

Chatbot CODRescue avec RAG (n8n + PostgreSQL)

Structure des Fichiers

```
chatbot/
├── n8n_workflows/
│   ├── chatbot_workflow_with_rag.json      # ★ Workflow n8n principal
│   ├── GUIDE_DEMARRAGE_RAPIDE.md          # 🚀 Commencez ici !
│   ├── README_RAG_Workflow.md            # 📖 Documentation complète
│   ├── PostgreSQL_Configuration.md        # 🔑 Configuration PostgreSQL
│   ├── Prompt_User_Message.md            # 💬 Configuration du prompt
│   └── SQL_Queries_Library.md             # 📊 25+ requêtes SQL prêtes
├── docker-compose.yml                     # 🐳 Configuration Docker n8n
├── .env.example                           # ⚙️ Variables d'environnement
├── services.py                            # 🔑 Service Django (legacy)
├── views.py                               # 🌐 Vues Django
├── n8n_client.py                          # 🔌 Client n8n
└── README_CHATBOT.md                     # 📖 Ce fichier
```

Démarrage Rapide

1. Démarrer n8n

```
cd chatbot
docker compose up -d
```

2. Accéder à n8n

Ouvrez <http://localhost:5779>

3. Configurer les Credentials

- **Google Gemini** : Obtenez une API Key sur <https://aistudio.google.com/app/apikey>
- **PostgreSQL** : Configurez avec vos paramètres de base de données

4. Importer le Workflow

1. Dans n8n : "+" → **"Import from File"**
2. Sélectionnez `n8n_workflows/chatbot_workflow_with_rag.json`

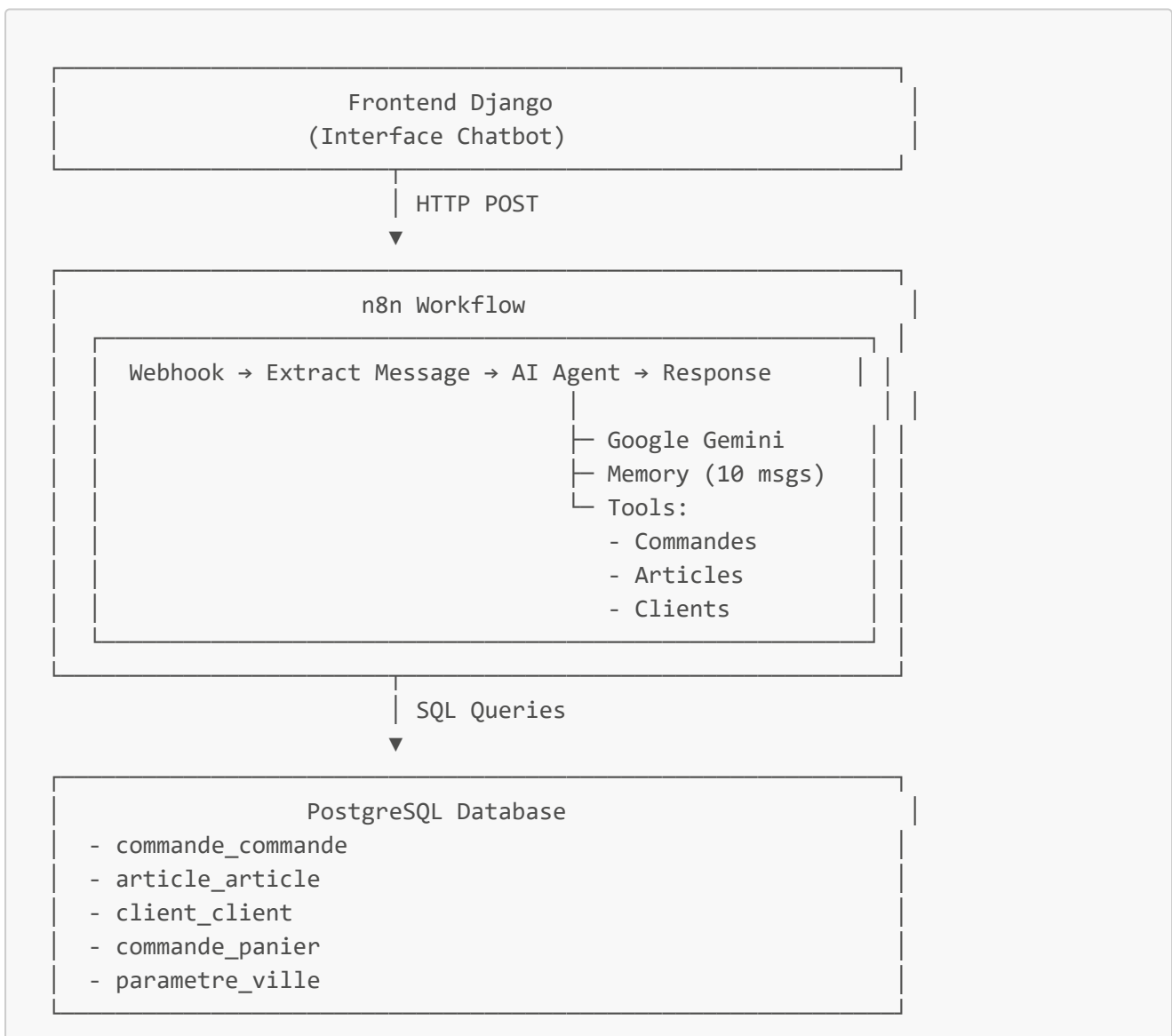
5. Activer et Tester

```
curl -X POST http://localhost:5779/webhook/chatbot \
  -H "Content-Type: application/json" \
  -d '{"message": "Combien de commandes aujourd\'hui ?", "sessionId": "test"}'
```

Documentation

Fichier	Description
GUIDE_DEMARRAGE_RAPIDE.md	Guide en 5 étapes pour démarrer
README_RAG_Workflow.md	Architecture, configuration, tests
PostgreSQL_Configuration.md	Configuration PostgreSQL, requêtes SQL
SQL_Queries_Library.md	25+ requêtes SQL prêtes à l'emploi
Prompt_User_Message.md	Configuration du prompt utilisateur

Architecture



✦ Fonctionnalités

☑ Actuellement Implémenté

- 📊 **Statistiques Commandes** : CA, nombre, panier moyen
- 🔍 **Recherche Commande** : Par numéro (OC-xxx, ADMIN-xxx, etc.)
- 📁 **Statistiques Articles** : Stock, valeur, actifs/inactifs
- 💬 **Mémoire de Conversation** : 10 derniers messages par session
- 🗣️ **IA Gemini 2.0 Flash** : Réponses intelligentes et contextuelles

🔗 Exemples de Questions

- ☑ "Combien de commandes aujourd'hui ?"
- ☑ "Quel est le chiffre d'affaires du mois ?"
- ☑ "Montre-moi la commande OC-00123"
- ☑ "Combien d'articles actifs ?"
- ☑ "Quelle est la valeur du stock ?"
- ☑ "Liste des commandes non payées"

🚀 À Ajouter (Requêtes SQL Disponibles)

- 👤 Recherche et statistiques clients
- 🚚 Suivi des livraisons
- 📊 KPIs avancés (évolution CA, taux de conversion)
- ⌚ Top clients, top articles
- 🌐 Analyse par ville/région
- 💰 Gestion des remises et paniers

🔑 Configuration

Variables d'Environnement (.env)

```
# n8n Configuration
N8N_HOST=0.0.0.0
N8N_PORT=5678
N8N_PROTOCOL=http
N8N_BASIC_AUTH_ACTIVE=true
N8N_BASIC_AUTH_USER=admin
N8N_BASIC_AUTH_PASSWORD=votre_mot_de_passe

# Webhook URLs
EDITOR_BASE_URL=http://localhost:5779
WEBHOOK_URL=http://localhost:5779

# PostgreSQL (pour référence)
DB_HOST=localhost
```

```
DB_PORT=5432
DB_NAME=codrescue_db
DB_USER=postgres
DB_PASSWORD=votre_mot_de_passe
```

Ports Utilisés

- **5779** : n8n (interface web + webhooks)
- **5432** : PostgreSQL
- **8000** : Django (runserver)



Tests

Test 1: Webhook Direct

```
curl -X POST http://localhost:5779/webhook/chatbot \
-H "Content-Type: application/json" \
-d '{
  "message": "Combien de commandes aujourd'hui ?",
  "sessionId": "test-session-001"
}'
```

Test 2: Depuis Django

```
import requests

response = requests.post(
    'http://localhost:5779/webhook/chatbot',
    json={
        'message': 'Quel est le CA du mois ?',
        'sessionId': f'user-{request.user.id}'
    }
)
print(response.json())
```

Test 3: Frontend

Le frontend est déjà implémenté dans votre application Django.
Mettez à jour l'URL du webhook dans la configuration.



Ajouter un Nouveau Tool

Exemple: Recherche Client

1. Créer le Tool Node

```
{
  "name": "search_client",
  "description": "Recherche un client par téléphone",
  "inputSchema": {
    "type": "object",
    "properties": {
      "telephone": {"type": "string"}
    },
    "required": ["telephone"]
  }
}
```

2. Créer le PostgreSQL Node

```
SELECT
  cl.nom || ' ' || cl.prenom as client,
  cl.telephone,
  COUNT(c.id) as nb_commandes,
  SUM(c.total_cmd) as total_achats
FROM client_client cl
LEFT JOIN commande_commande c ON c.client_id = cl.id
WHERE cl.telephone LIKE '%{{ $json.telephone }}%'
GROUP BY cl.id, cl.nom, cl.prenom, cl.telephone
LIMIT 5;
```

3. Connecter au AI Agent

Connectez le Tool via la connexion `ai_tool`

4. Tester

```
curl -X POST http://localhost:5779/webhook/chatbot \
  -H "Content-Type: application/json" \
  -d '{"message": "Recherche le client 0612345678", "sessionId": "test"}'
```

Sécurité

Important pour la Production

1. Authentification :

- Basic Auth activé dans docker-compose.yml
- Ajoutez un token API pour le webhook
- Considérez OAuth2 pour plus de sécurité

2. PostgreSQL :

- Utilisez un utilisateur en lecture seule
- Ne donnez JAMAIS accès en écriture (INSERT, UPDATE, DELETE)
- Limitez les résultats avec LIMIT

3. HTTPS :

- Utilisez ngrok ou un reverse proxy (nginx)
- Configurez SSL/TLS en production

4. Rate Limiting :

- Limitez le nombre de requêtes par utilisateur
- Implémentez des quotas

Dépannage

Problème: "Connection refused" (PostgreSQL)

➡ Si n8n est dans Docker, utilisez `host.docker.internal` au lieu de `localhost`

Problème: "Credential not found"

➡ Configurez vos credentials Google Gemini et PostgreSQL dans n8n

Problème: "Tool execution failed"

➡ Vérifiez les requêtes SQL dans pgAdmin

➡ Regardez les logs dans n8n (onglet "Executions")

Problème: Le chatbot ne répond pas

➡ Vérifiez que le workflow est **activé**

➡ Testez le webhook avec curl

➡ Regardez les logs d'exécution

Métriques & Monitoring

Logs n8n

- Interface : Onglet "Executions"
- Voir les requêtes, réponses, erreurs

Logs PostgreSQL

```
# Linux
tail -f /var/log/postgresql/postgresql-*.log
```

```
# Ou dans psql
SELECT * FROM pg_stat_activity;
```

Performance

- Temps de réponse moyen : < 2 secondes
- Requêtes SQL : < 100ms
- Gemini API : < 1 seconde

Roadmap

Phase 1 : Actuellement ☒

- ☒ Workflow n8n avec RAG
- ☒ Statistiques commandes
- ☒ Recherche commande
- ☒ Statistiques articles
- ☒ Mémoire de conversation

Phase 2 : Court Terme

- ☐ Recherche et statistiques clients
- ☐ Suivi des livraisons
- ☐ KPIs avancés
- ☐ Top clients/articles
- ☐ Analyse géographique

Phase 3 : Long Terme

- ☐ Recommandations de produits
- ☐ Prédiction de churn
- ☐ Alertes automatiques
- ☐ Dashboards visuels
- ☐ Export de rapports

Support

Documentation

- n8n : <https://docs.n8n.io>
- Google Gemini : <https://ai.google.dev/docs>
- PostgreSQL : <https://www.postgresql.org/docs/>

Fichiers de Référence

- [GUIDE_DEMARRAGE_RAPIDE.md](#) - Démarrage en 5 étapes
- [SQL_Queries_Library.md](#) - 25+ requêtes SQL
- [README_RAG_Workflow.md](#) - Documentation complète

Félicitations !

Vous avez maintenant un chatbot intelligent avec RAG qui peut :

- ☒ Interroger votre base de données PostgreSQL
- ☒ Répondre aux questions sur les commandes, articles, clients
- ☒ Fournir des statistiques en temps réel
- ☒ Garder l'historique des conversations
- ☒ S'intégrer avec votre frontend Django

Bon développement ! 

Dernière mise à jour : 2025-12-03